

专题讲义：BidKV 与状态管理驱动的推理系统优化

LLM Inference Infrastructure - 2026

从 vLLM 的连续批处理与 KV cache 出发，本讲义把在线推理服务理解为状态化系统。

核心线索是：如何观测状态压力，如何管理多级记忆，如何让调度器根据状态价值做执行决策。

材料结合 BidKV (SC 2026) 与 vLLM-HUST 系列实践，面向课程讨论、实验设计和项目选题。

本专题的三个问题

- 状态压力从哪里来？
- 哪些 KV 状态值得保留？
- 怎样设计可复现的性能比较？

1. 为什么继续讨论推理系统优化

vLLM 让连续批处理、PagedAttention 和 OpenAI-compatible serving 成为推理系统的基础能力。课程后半段需要进一步回答一个问题：当服务进入长上下文、多轮对话、工具调用、MoE 和异构硬件场景后，性能瓶颈为什么不再只是一次 kernel 或一次 batch 的吞吐，而是“状态如何被保留、迁移、复用和回收”。

本专题围绕 BidKV: Utility-Guided Preemption Scheduling for KV-Pressure LLM Serving (SC 2026) 以及 vLLM-HUST 系列实践，介绍一种更适合推理基础设施的分析视角：把在线推理看成状态化系统，而不是单次请求执行器。

2. 统一视角：推理系统是状态化系统

推理请求在系统中留下大量可复用或必须管理的状态，包括 prompt tokens、KV cache blocks、prefix tree、adapter / LoRA 状态、MoE expert routing、设备内外的 memory pages，以及 benchmark 中记录的 workload profile。

这些状态决定了三类关键问题：

- 观测：系统是否知道当前状态压力来自哪里。
- 管理：系统是否能在 GPU / NPU memory、host memory、storage 和 network 之间稳定迁移状态。
- 执行：调度器是否能根据状态价值决定先执行谁、保留谁、抢占谁。

课程中可以把它概括为“观测 - 管理 - 执行”的闭环。

3. BidKV：KV 压力下的 utility-guided preemption

长上下文和高并发会让 KV cache 成为服务系统最紧张的资源之一。传统抢占策略往往把请求看成相似对象，只根据到达时间、队列位置或粗粒度资源占用做决策。BidKV 的核心问题是：在 KV 压力很高时，哪些请求的 KV 状态更值得保留，哪些请求可以更低代价地被抢占或恢复。

BidKV 引入 utility-guided preemption scheduling。它不是简单追求“少抢占”，而是评估不同请求的状态价值和抢占代价，让调度器在资源紧张时做更接近服务目标的取舍。

可以从四个维度理解这个设计：

- 状态价值：当前 KV 是否已经沉淀了大量计算，丢弃后重算成本有多高。
- 未来收益：保留该请求是否有助于更快完成后续 decode。
- 资源压力：该请求占用的 KV block 是否正在阻塞其他请求进入系统。
- 服务目标：吞吐、尾延迟、公平性和任务完成率之间如何权衡。

4. 技术线一：状态感知的调度与资源治理

面向长序列、agent workload 和 MoE 服务，调度器需要感知更多状态：

- 请求处于 prefill、decode、等待工具返回还是等待资源恢复。
- KV cache 的占用是否由少数长请求主导。
- 某个请求被暂停后，恢复代价是重新 prefill、迁移 KV，还是仅等待下一轮 decode。
- 不同 workload 对 TTFT、TPOT、吞吐和完成率的敏感度是否相同。

课程讨论建议：让学生实现一个简化版调度模拟器，对比 FIFO、最短剩余 decode、最大 KV 占用优先抢占和 utility-guided 抢占。

5. 技术线二：硬件感知的多级记忆管理与语义一致性

KV cache 管理不是单纯的内存分配问题。它同时受硬件、runtime 和语义一致性约束：

- 硬件侧：HBM / DDR / host memory / storage 的容量、带宽和迁移成本不同。
- Runtime 侧：block size、attention backend、graph capture、enforce-eager、数据类型会改变性能曲线。
- 语义侧：prefix reuse、分层缓存、adapter 状态和多轮会话必须保证读写一致。

因此，优化不应只报告“某个版本更快”，还要说明参数、模型、芯片、并发、输入输出长度、是否启用 graph、是否复用 prefix 等条件是否一致。

6. 技术线三：状态复用驱动的长序列与 MoE 推理加速

很多真实工作负载并不是独立随机请求。代码生成、文档问答、agent-research、多轮对话和视觉问答都有可复用状态：

- prefix 或 system prompt 复用。
- 文档 chunk、retrieval context 和工具返回结果复用。
- MoE 中 expert routing 与后端适配路径复用。
- 长序列任务中局部窗口、摘要状态和历史 KV 的分层复用。

这条技术线的关键是把“复用机会”变成系统可观测的对象，再把它接入调度、缓存淘汰和 benchmark 设计。

7. 从 benchmark 中学到的方法论

vLLM-HUST benchmark

的经验说明，性能结论只有在实验条件对齐时才可靠。课程实验中要特别检查：

- workload 是否一致：random、ShareGPT、agent、code、vision、throughput 与 latency 模式不能混在一起比较。
- 模型是否一致：dense、MoE、VL、INT8 / FP16 等不能只看名字相近。
- 硬件是否一致：910B2、910B3、多卡拓扑和驱动栈必须明确标注。
- 参数是否一致：input / output token size、concurrency、max-num-seqs、max-model-len、block size、enforce-eager 等会显著影响结果。
- 基线是否一致：比较 vLLM-HUST PR 时，应和同一条件下的 vLLM / vLLM-Ascend baseline 对比，而不是混用两个历史优化版本。

8. 建议课程项目

项目一：实现 KV pressure observer。记录每轮调度时的 KV block 占用、等待队列、抢占次数和恢复代价。

项目二：实现简化 BidKV simulator。输入请求长度分布和 KV 容量，比较不同抢占策略的吞吐、尾延迟和重算成本。

项目三：设计 prefix reuse benchmark。构造共享 system prompt、共享文档上下文和多轮会话三类负载，观察复用率如何影响吞吐和 TTFT。

项目四：做一次 benchmark anomaly

diagnosis。给定一组性能折线，判断突降是参数不一致、硬件差异、真实回归，还是缺少 baseline 数据。

9. 延伸阅读

- BidKV: Utility-Guided Preemption Scheduling for KV-Pressure LLM Serving, SC 2026。
- vLLM PagedAttention 与 continuous batching 相关文档。
- vLLM-Ascend / vLLM-HUST 的异构后端适配与 benchmark 记录。

10. 本讲义的学习目标

完成本专题后，学生应该能够：

- 用状态化系统视角解释在线 LLM serving 的瓶颈。
- 区分 KV cache 优化、请求调度优化和 benchmark 方法论问题。
- 设计一个能比较优化 PR 与 baseline 的实验矩阵。
- 判断性能折线中的提升、退化和缺失数据分别意味着什么。